

Dr. Christoph M.A.
Scheich

Angewandte Informatik
Hochschule Fulda

Christoph.scheich@infor-
matik.hs-fulda.de

Datenbanksystem
e

Dr. Christoph M.A.
Scheich

Angewandte Informatik
Hochschule Fulda

Christoph.scheich@infor-
matik.hs-fulda.de

SQL – Structured Query Language, Teil 2

- Decision-support Queries
- Quantifizierung
- Nullwerte
- Jointypen
- Rekursion
- Veränderungen am Datenbestand
- Sichten/Views
- Zugang zu DBMS Anwenderprogramme

Wiederholen Sie zuerst noch einmal sorgfältig die letzte Vorlesung. Es kommt hier jetzt auf die Details an

Decision-Support Anfrage mit geschachtelten Unterabfragen

```
select h.VorlNr, h.AnzProVorl, g.GesamtAnz,  
        h.AnzProVorl/g.GesamtAnz as Marktanteil  
  
from ( select VorlNr, count(*) as AnzProVorl  
        from hoeren  
        group by VorlNr ) h,  
      ( select count (*) as GesamtAnz  
        from Studenten) g;
```

```
select h.VorlNr, h.AnzProVorl, g.GesamtAnz,  
        cast(h.AnzProVorl as decimal(6,2)) / g.GesamtAnz  
as Marktanteil  
  
from ( select VorlNr, count(*) as AnzProVorl  
        from hoeren  
        group by VorlNr ) h,  
      ( select count (*) as GesamtAnz  
        from Studenten) g;
```

Ergebnis der Anfrage / Query

VorlNr	AnzProVorl	GesamtAnz	Marktanteil
4052	1	8	.125
5001	4	8	.5
5022	2	8	.25
...	...	...	...

Modularisierung der Query mit 'with'

```
with h as (select VorlNr, count(*) as AnzProVorl
           from hoeren
           group by VorlNr) ,
g as (select count (*) as GesamtAnz
     from Studenten)

select h.VorlNr, h.AnzProVorl, g.GesamtAnz,
       cast(h.AnzProVorl as decimal(6,2)) / g.GesamtAnz
       as Marktanteil
from g,h
```

Modularisierung der Query mit 'with', weiteres Beispiel

```
with kenntSich(ProfPersNr, StudMatrNr) as  
    (select distinct v.gelesenVon,h.MatrNr  
     from hoeren h join Vorlesungen v on h.VorlNr=v.VorlNr) ,  
     kenntAnzahl(ProfPersNr, AnzStudenten) as  
    (select ProfPersNr, count(*) as AnzStudenten  
     from kenntSich  
     group by ProfPersNr),  
    wieviele (GesamtAnz) as (select count(*) from Studenten)
```

```
select k.ProfPersNr, p.Name, k.AnzStudenten, w.GesamtAnz,  
1.00 * k.AnzStudenten/w.GesamtAnz as Bekanntheitsgrad  
from kenntAnzahl k, wieviele w, Professoren p  
where k.ProfPersNr = p.PersNr  
order by Bekanntheitsgrad desc
```

```
( select Name  
  from Assistenten )  
union  
( select Name  
  from Professoren );
```

```
select Name  
from Professoren  
where PersNr not in ( select gelesenVon  
                      from Vorlesungen );
```

Quantifizierte Anfragen in SQL - Existenz

- Existenzquantor:
exists select Name
from Professoren
where not exists (select *
from Vorlesungen
where gelesenVon = PersNr);

- SQL-92 hat keinen Allquantor
- Allquantifizierung muß also durch eine äquivalente Anfrage mit Existenzquantifizierung ausgedrückt werden
- Beispiel einer solchen Anfrage: Wer hat **alle** vierstündigen Vorlesungen gehört?

- SQL-Umsetzung folgt direkt:

select s.*

from Studenten s

where not exists

(select *

from Vorlesungen v

where v.SWS = 4 and not exists

(select *

from hoeren h

where h.VorlNr = v.VorlNr and h.MatrNr=s.MatrNr));

Wo keine
unbesuchten 4er
Vorlesungen
existieren



Ab hier: Menge der
'unbesuchten' 4er
Vorlesungen pro
Student

- Allquantifizierung kann immer auch durch eine **count**-Aggregation ausgedrückt werden (als eine Art Workaround)
- Wir betrachten dazu eine etwas einfachere Anfrage, in der wir die (*MatrNr* der) Studenten ermitteln wollen, die *alle* Vorlesungen hören:

select h.MatrNr

from hoeren h

group by h.MatrNr

having count (*) = (**select** count (*) **from** Vorlesungen);

- Wie formuliert man die komplexere Anfrage: Wer hat alle vierstündigen Vorlesungen gehört
- Grundidee besteht darin, vorher durch einen Join die Studenten/Vorlesungs-Paare einzuschränken und danach das Zählen durchzuführen

- Wie formuliert man die komplexere Anfrage: Wer hat alle vierstündigen Vorlesungen gehört
- Grundidee besteht darin, vorher durch einen Join die Studenten/Vorlesungs-Paare einzuschränken und danach das Zählen durchzuführen

```
select s.MatrNr, s.name, count(*) as VierSWSInst
from Vorlesungen v, hoeren h, Studenten s
where v.SWS = 4 and v.VorlNr = h.VorlNr and h.MatrNr = s. MatrNr
group by s.MatrNr, s.name
having count(*) = (select count(*) from Vorlesungen
where SWS = 4)
```

- Unbekannter Wert
- Wird vielleicht später nachgereicht
- Nullwerte können auch im Zuge der Abfrageauswertung entstehen (Bsp. äußere Joins)
- Manchmal sehr überraschende Abfrageergebnisse, wenn Nullwerte vorkommen

select count (*)

from Studenten

where Semester < 13 **or** Semester > =13

- Wenn es Studenten gibt, deren *Semester*-Attribut den Wert **null** hat, werden diese nicht mitgezählt
- Der Grund liegt in folgenden Regeln für den Umgang mit **null**-Werten begründet:

1. In arithmetischen Ausdrücken werden Nullwerte propagiert, d.h. sobald ein Operand **null** ist, wird auch das Ergebnis **null**. Dementsprechend wird z.B. **null** + 1 zu **null** ausgewertet- aber auch **null** * 0 wird zu **null** ausgewertet.
2. SQL hat eine dreiwertige Logik, die nicht nur **true** und **false** kennt, sondern auch einen dritten Wert **unknown**. Diesen Wert liefern Vergleichsoperationen zurück, wenn mindestens eines ihrer Argumente **null** ist. Beispielsweise wertet SQL das Prädikat (*PersNr=...*) immer zu **unknown** aus, wenn die *PersNr* des betreffenden Tupels den Wert **null** hat.
3. Logische Ausdrücke werden nach den folgenden Tabellen berechnet:

Diese Berechnungsvorschriften sind recht intuitiv. Unknown or true wird z.B. zu **true** - die Disjunktion ist mit dem **true**-Wert des rechten Arguments immer erfüllt, unabhängig von der Belegung des linken Arguments. Analog ist **unknown and false** automatisch **false** - keine Belegung des linken Arguments könnte die Konjunktion mehr erfüllen.

4. In einer **where**-Bedingung werden nur Tupel weitergereicht, für die die Bedingung **true** ist. Insbesondere werden Tupel, für die die Bedingung zu **unknown** ausgewertet, nicht ins Ergebnis aufgenommen.
5. Bei einer Gruppierung wird **null** als ein eigenständiger Wert aufgefaßt und in eine eigene Gruppe eingeordnet.

not	
true	false
unknown	unknown
false	true

and	true	unknown	false
true	true	unknown	false
unknown	unknown	unknown	false
false	false	false	false

or	true	unknown	false
true	true	true	true
unknown	true	unknown	unknown
false	true	unknown	false

```
select *  
from Studenten  
where Semester > = 1 and Semester < = 4;
```

```
select *  
from Studenten  
where Semester between 1 and 4;
```

```
select *  
from Studenten  
where Semester in (1,2,3,4);
```

Platzhalter "%" ; "_"

- "%" steht für beliebig viele (auch gar kein) Zeichen
- "_" steht für genau ein Zeichen

select *

from Studenten

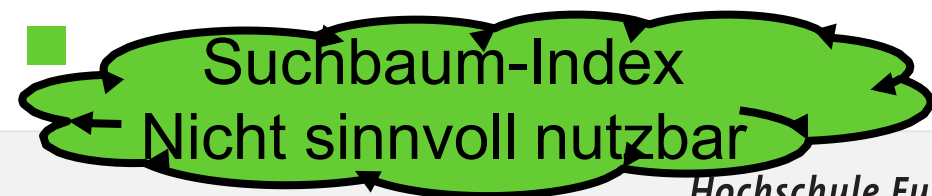
where Name **like** 'T%eophrastos';

select distinct s.Name

from Vorlesungen v, hoeren h, Studenten s

where s.MatrNr = h.MatrNr **and** h.VorlNr = v.VorlNr **and**

v.Titel **like** '%thik%';



```
select MatrNr, ( case when Note < 1.5 then 'sehr gut'  
                  when Note < 2.5 then 'gut'  
                  when Note < 3.5 then 'befriedigend'  
                  when Note < 4.0 then 'ausreichend'  
                  else 'nicht bestanden'  
                end)  
from pruefen;
```

- Die **erste** qualifizierende **when**-Klausel wird ausgeführt

- **cross join:** Kreuzprodukt
- **natural join:** natürlicher Join (nicht alle DBMS)
- Join oder inner join: Theta-Join
- left, right oder full outer join: äußerer Join
- union join: Vereinigungs-Join (wird hier nicht vorgestellt)

select *
from R_1, R_2
where $R_1.A = R_2.B;$

select *
from R_1 **join** R_2 **on** $R_1.A = R_2.B;$

Query Plans 1

The screenshot shows the Oracle SQL Developer interface. The left pane displays the 'Connections' tree with 'DBDMF20' selected. The main window shows the 'Worksheet' tab with the following SQL code:

```
explain plan for
select * from professoren;

select plan_table_output from table(dbms_xplan.display());
```

The 'Query Result' tab shows the execution results. The first row indicates the plan hash value: 4240101377. The subsequent rows show the execution plan details:

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		8	144	3 (0)	00:00:01
1	TABLE ACCESS FULL	PROFESSOREN	8	144	3 (0)	00:00:01

The 'Messages - Log' pane at the bottom shows the following message:

```
Mising skipWhiteSpaceBeforeNode.; had to reset format program
```

Query Plans 2

Oracle SQL Developer : DBDMF20

Connections

- Oracle Connections
 - BIGDATAD20
 - DBDMA01
 - DBDMF20
 - Tables (Filtered)
 - Views
 - Indexes
 - Packages
 - Procedures
 - Functions
 - Operators
 - Queues
 - Queues Tables
 - Triggers

Reports

- All Reports
 - Analytic View Reports
 - Data Dictionary Reports
 - Data Modeler Reports
 - OLAP Reports
 - TimesTen Reports
 - User Defined Reports

Worksheet

```
explain plan for
select * from studenten, hoeren;

select plan_table_output from table(dbms_xplan.display());
```

Script Output x Query Result x

All Rows Fetched: 11 in 0.018 seconds

PLAN_TABLE_OUTPUT

1 Plan hash value: 3328111413

2									
3									
4	Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time		
5									
6	0	SELECT STATEMENT		104	2808	7 (0)	00:00:01		
7	1	MERGE JOIN CARTESIAN		104	2808	7 (0)	00:00:01		
8	2	TABLE ACCESS FULL	STUDENTEN	8	144	3 (0)	00:00:01		
9	3	BUFFER SORT		13	117	4 (0)	00:00:01		
10	4	INDEX FAST FULL SCAN	SYS_C0016592	13	117	1 (0)	00:00:01		
11									

Messages - Log

Mising skipWhiteSpaceBeforeNode;; had to reset format program

Query Plans 3

Connections

- Oracle Connections
 - BIGDATAD20
 - DBDMA01
 - DBDMF20
 - Tables (Filtered)
 - Views
 - Indexes
 - Packages
 - Procedures
 - Functions
 - Operators
 - Queues
 - Queues Tables
 - Triggers
 - Types
 - Sequences

Reports

- All Reports
- Analytic View Reports
- Data Dictionary Reports
- Data Modeler Reports
- OLAP Reports
- TimesTen Reports
- User Defined Reports

DBDMF207.sql Welcome Page DBDMF20

Worksheet Query Builder

```
explain plan for
select * from studenten, hoeren
where studenten.matrnr = hoeren.matrnr;

select plan_table_output from table(dbms_xplan.display());
```

Script Output Query Result

SQL All Rows Fetched: 18 in 0.021 seconds

PLAN_TABLE_OUTPUT

1 Plan hash value: 158289854

2

3

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		13	351	4 (25)	00:00
1	MERGE JOIN		13	351	4 (25)	00:00
2	TABLE ACCESS BY INDEX ROWID	STUDENTEN	8	144	2 (0)	00:00
3	INDEX FULL SCAN	SYS_C0016582	8		1 (0)	00:00
* 4	SORT JOIN		13	117	2 (50)	00:00
5	INDEX FULL SCAN	SYS_C0016592	13	117	1 (0)	00:00

12

13

14 Predicate Information (identified by operation id):

Messages - Log

Query Plans 4

The screenshot displays the Oracle SQL Developer interface. The 'Connections' pane on the left shows a tree view of database objects for the 'DBDMF20' connection. The main window is split into a 'Worksheet' and a 'Query Builder'. The 'Worksheet' contains the following SQL code:

```
explain plan for
select * from studenten join hoeren on studenten.matrnr = hoeren.matrnr;

select plan_table_output from table(dbms_xplan.display());
```

The 'Query Result' pane shows the output of the second query, which is the execution plan for the first query. The output is as follows:

Script Output * Query Result *
All Rows Fetched: 18 in 0.099 seconds

PLAN_TABLE_OUTPUT

1 Plan hash value: 158289854

2

3

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		13	351	4 (25)	00:00
1	MERGE JOIN		13	351	4 (25)	00:00
2	TABLE ACCESS BY INDEX ROWID	STUDENTEN	8	144	2 (0)	00:00
3	INDEX FULL SCAN	SYS_C0016582	8		1 (0)	00:00
* 4	SORT JOIN		13	117	2 (50)	00:00
5	INDEX FULL SCAN	SYS_C0016592	13	117	1 (0)	00:00

12

13

14 Predicate Information (identified by operation id):

Messages - Log

Beobachtung

- Kein Cross-Join
- Gleiche Ausführung in alter und neuer Join-Syntax

SQL Developer interface showing a query plan for a query using the old 'where' join syntax. The query is:

```
explain plan for  
select * from studenten, hoeren  
where studenten.matnr = hoeren.matnr;  
select plan_table_output from table(dbms_xplan.display());
```

The execution plan (PLAN_TABLE_OUTPUT) is as follows:

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		13	351	4 (25)	00:00
1	MERGE JOIN		13	351	4 (25)	00:00
2	TABLE ACCESS BY INDEX ROWID	STUDENTEN	8	144	2 (0)	00:00
3	INDEX FULL SCAN	SYS_C0016582	8	1	1 (0)	00:00
4	SORT JOIN		13	117	2 (50)	00:00
5	INDEX FULL SCAN	SYS_C0016592	13	117	1 (0)	00:00

Predicate Information (identified by operation id):

SQL Developer interface showing a query plan for a query using the new 'join on' syntax. The query is:

```
explain plan for  
select * from studenten join hoeren on studenten.matnr = hoeren.matnr;  
select plan_table_output from table(dbms_xplan.display());
```

The execution plan (PLAN_TABLE_OUTPUT) is as follows:

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		13	351	4 (25)	00:00
1	MERGE JOIN		13	351	4 (25)	00:00
2	TABLE ACCESS BY INDEX ROWID	STUDENTEN	8	144	2 (0)	00:00
3	INDEX FULL SCAN	SYS_C0016582	8	1	1 (0)	00:00
4	SORT JOIN		13	117	2 (50)	00:00
5	INDEX FULL SCAN	SYS_C0016592	13	117	1 (0)	00:00

Predicate Information (identified by operation id):

select p.PersNr, p.Name, f.PersNr, f.Note, f.MatrNr,
s.MatrNr, s.Name

from Professoren p **left outer join**

(prüfen f **left outer join** Studenten s **on**
f.MatrNr= s.MatrNr)

on p.PersNr=f.PersNr;

PersNr	p.Name	f.PersNr	f.Note	f.MatrNr	s.MatrNr	s.Name
2126	Russel	2126	1	28106	28106	Carnap
2125	Sokrates	2125	2	25403	25403	Jonas
2137	Kant	2137	2	27550	27550	Schopen- hauer
2136	Curie	-	-	-	-	-
...	...	...	...	...	...	...

```
select p.PersNr, p.Name, f.PersNr, f.Note, f.MatrNr,  
        s.MatrNr, s.Name
```

```
from Professoren p right outer join
```

```
    (pruefen f right outer join Studenten s on  
      f.MatrNr= s.MatrNr)
```

```
on p.PersNr=f.PersNr;
```

PersNr	p.Name	f.PersNr	f.Note	f.MatrNr	s.MatrNr	s.Name
2126	Russel	2126	1	28106	28106	Carnap
2125	Sokrates	2125	2	25403	25403	Jonas
2137	Kant	2137	2	27550	27550	Schopen- hauer
-	-	-	-	-	26120	Fichte
...	...	...	...	...	...	...

```
select p.PersNr, p.Name, f.PersNr, f.Note, f.MatrNr,  
        s.MatrNr, s.Name  
from Professoren p full outer join  
        (pruefen f full outer join Studenten s on  
         f.MatrNr= s.MatrNr)  
on p.PersNr=f.PersNr;
```

Äußere Joins – Full Outer

p.PersNr	p.Name	f.PersNr	f.Note	f.MatrNr	s.MatrNr	s.Name
2126	Russel	2126	1	28106	28106	Carnap
2125	Sokrates	2125	2	25403	25403	Jonas
2137	Kant	2137	2	27550	27550	Schopen- hauer
-	-	-	-	-	26120	Fichte
...	...	...	...	...	...	...
2136	Curie	-	-	-	-	-
...	...	...	...	...	...	...

```
select Vorgaenger  
from voraussetzen, Vorlesungen  
where Nachfolger= VorlNr and  
        Titel= 'Der Wiener Kreis'
```

voraussetzen	
Vorgänger	Nachfolger
5001	5041
5001	5043
5001	5049
5041	5216
5043	5052
5041	5052
5052	5259

Vorlesungen			
VorlNr	Titel	SW S	gelesenV on
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

Der Wiener Kreis
5259

Wissenschaftstheorie
5052

Bioethik
5216

Erkenntnistheorie
5043

Ethik
5041

Mäeutik
5049

Grundzüge
5001

Vor-Vorgänger-Vorlesungen des 'Wiener Kreises'

```
select v1.Vorgaenger  
from voraussetzen v1, voraussetzen v2, Vorlesungen v  
where v1.Nachfolger= v2.Vorgaenger and  
        v2.Nachfolger= v.VorlNr and  
        v.Titel='Der Wiener Kreis'
```

select v1.Vorgaenger

from voraussetzen v1, voraussetzen v2, Vorlesungen v

where v1.Nachfolger= v2.Vorgaenger **and**

v2.Nachfolger= v.VorlNr **and**

v.Titel='Der Wiener Kreis'

voraussetzen	
Vorgänger	Nachfolger
5001	5041
5001	5043
5001	5049
5041	5216
5043	5052
5041	5052
5052	5259

voraussetzen	
Vorgänger	Nachfolger
5001	5041
5001	5043
5001	5049
5041	5216
5043	5052
5041	5052
5052	5259

Vorlesungen			
VorlNr	Titel	SWS	gelesen V on
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

```
select v1.Vorgänger
from voraussetzen v1
    ...
    voraussetzen vn_minus_1
    voraussetzen vn,
    Vorlesungen v
where v1.Nachfolger= v2.Vorgänger and
    ...
    vn_minus_1.Nachfolger= vn.Vorgänger and
    vn.Nachfolger = v.VorlNr and
    v.Titel= 'Der Wiener Kreis'
```

Mathematische Definition [\[Bearbeiten | Quelltext bearbeiten \]](#)

Die transitive Hülle $t(R) \equiv R^+$ einer zweistelligen Relation R auf einer Menge M ist gegeben durch:

$$x \, t(R) \, y \Leftrightarrow x \, R^+ \, y \Leftrightarrow \exists n \geq 0 \, \exists x_1, \dots, x_n \in M : x \, R \, x_1 \, R \, x_2 \, R \dots \, R \, x_n \, R \, y^{[1]}$$

Beispiele [\[Bearbeiten | Quelltext bearbeiten \]](#)

- Ist R gegeben durch die zwei Paare (a, b) und (b, c) , dann enthält R^+ zusätzlich das Paar (a, c) . Für R^* kommen die weiteren Paare (a, a) , (b, b) und (c, c) dazu.

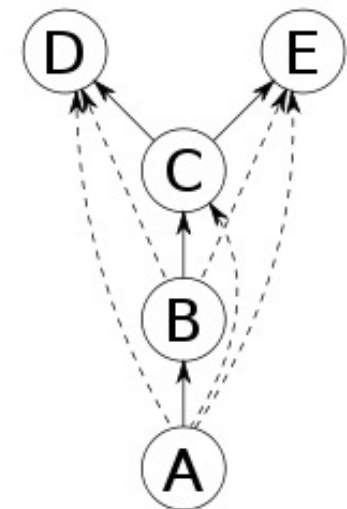
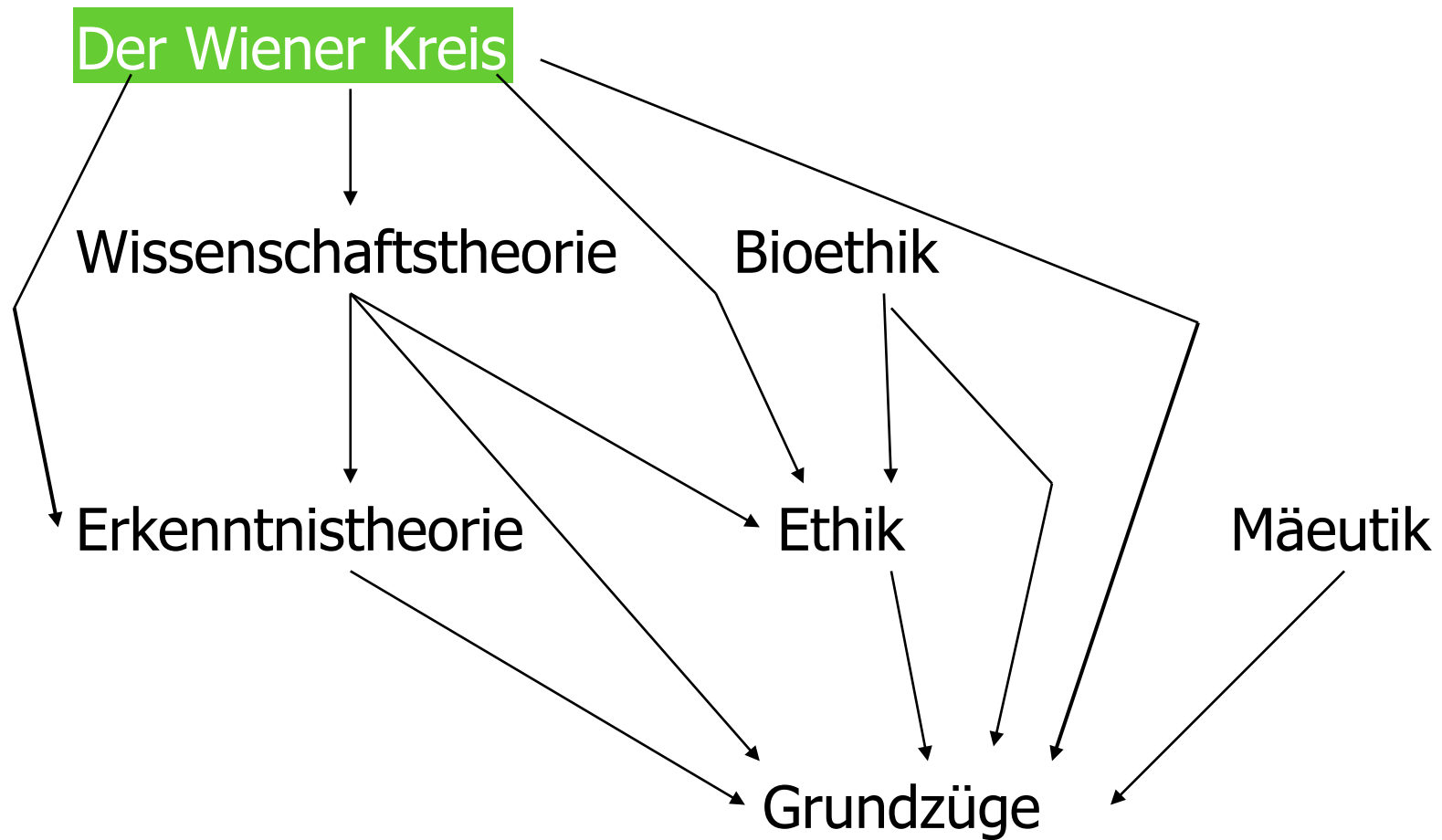


Illustration des Beispiels:
durchgezogene Pfeile zeigen direkte
Beziehungen an, gestrichelte Pfeile die
in der transitiven Hülle dazu
kommenden Relationen



select Titel

from Vorlesungen

where VorlNr **in** (**select** Vorgaenger

from voraussetzen

connect by Nachfolger= **prior** Vorgaenger

start with Nachfolger= (**select** VorlNr

from Vorlesungen

where Titel= 'Der
Wiener Kreis'));

Funktioniert nicht
für alle
Datenbanken

Oracle-spezifisch

voraussetzen	
Vorgänger	Nachfolger
5001	5041
5001	5043
5001	5049
5041	5216
5043	5052
5041	5052
5052	5259

Vorlesungen			
VorlNr	Titel	SWS	gelesen V on
5001	Grundzüge	4	2137
5041	Ethik	4	2125
5043	Erkenntnistheorie	3	2126
5049	Mäeutik	2	2125
4052	Logik	4	2125
5052	Wissenschaftstheorie	3	2126
5216	Bioethik	2	2126
5259	Der Wiener Kreis	2	2133
5022	Glaube und Wissen	2	2134
4630	Die 3 Kritiken	4	2137

Der Wiener Kreis
5259

Wissenschaftstheorie
5052

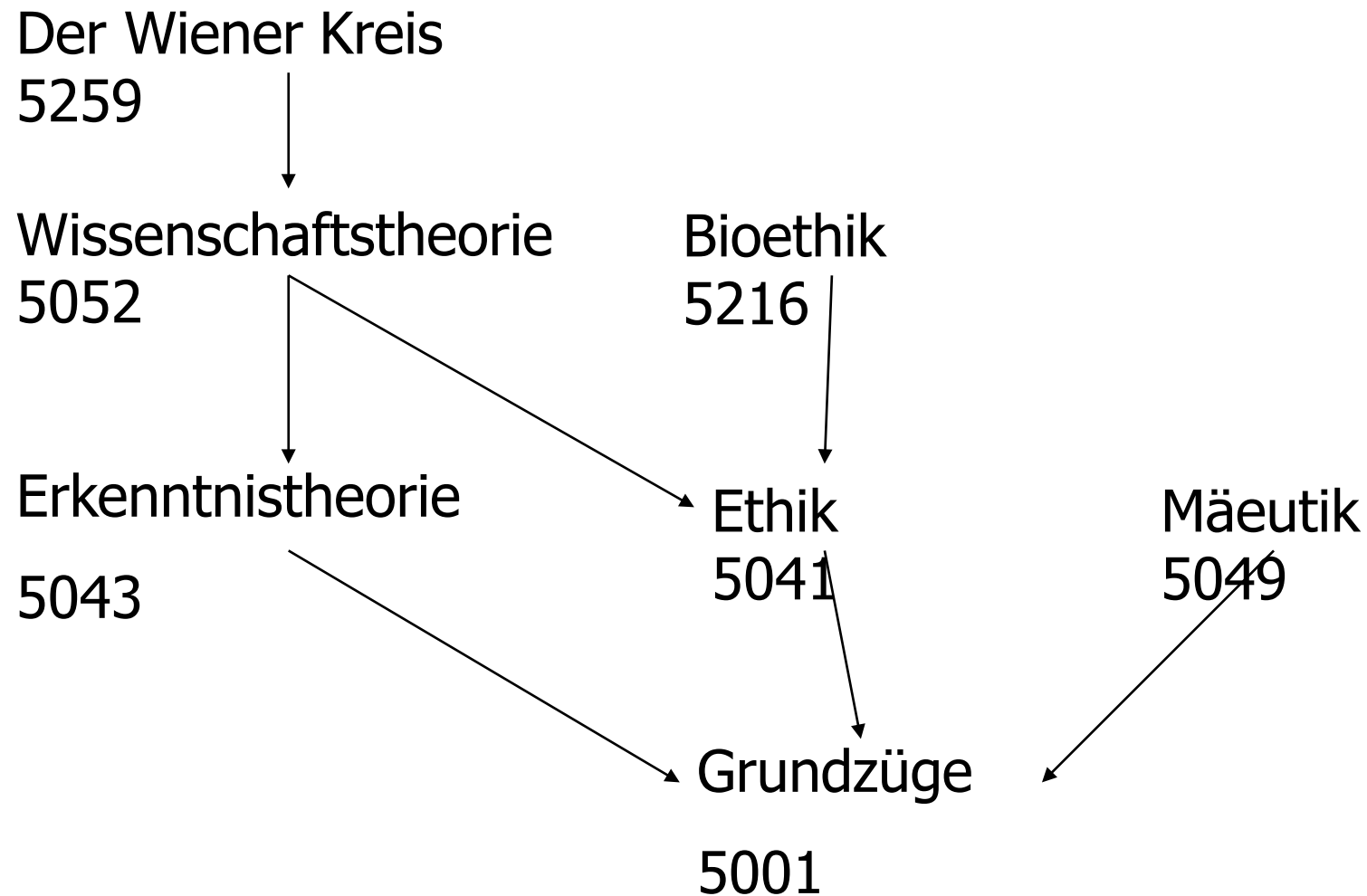
Bioethik
5216

Erkenntnistheorie
5043

Ethik
5041

Mäeutik
5049

Grundzüge
5001



Wissenschaftstheorie
Ethik
Erkenntnistheorie
Grundzüge

Gleiche Abfrage: Vorgänger-Vorlesungen des 'Wiener Kreises' der tiefe n

```
with TransVorl (Vorg, Nachf)
as (select Vorgaenger, Nachfolger from voraussetzen
      union all
      select t.Vorg, v.Nachfolger
      from TransVorl t, voraussetzen v
      where t.Nachf= v.Vorgaenger)

select Titel from Vorlesungen where VorlNr in
  (select Vorg from TransVorl where Nachf in
    (select VorlNr from Vorlesungen
      where Titel= `Der Wiener Kreis` ) )
```

- Zuerst wird eine temporäre Sicht *TransVorl* mit der **with-**Klausel angelegt
- Diese Sicht *TransVorl* ist rekursiv definiert, da sie selbst in der Definition vorkommt
- Aus dieser Sicht werden dann die gewünschten Tupel extrahiert
- Ergebnis ist natürlich wie gehabt

Einfügen von Tupeln

insert into hoeren

select MatrNr, VorlNr

from Studenten, Vorlesungen

where Titel= `Logik`;

insert into Studenten (MatrNr, Name)

values (28121, `Archimedes`);

Veränderungen am Datenbestand – Nochmal im Detail

Studenten		
MatrNr	Name	Semester
...	...	...
29120	Theophrastos	2
29555	Feuerbach	2
28121	Archimedes	-

Löschen von Tupeln

```
delete Studenten  
where Semester > 13;
```

Verändern von Tupeln

```
update Studenten  
set Semester = Semester + 1;
```

für den Datenschutz

create view prüfenSicht as

select MatrNr, VorlNr, PersNr

from prüfen

für den Datenschutz

create view prüfenSicht as

select MatrNr, VorlNr, PersNr
from prüfen

Statistische Sicht

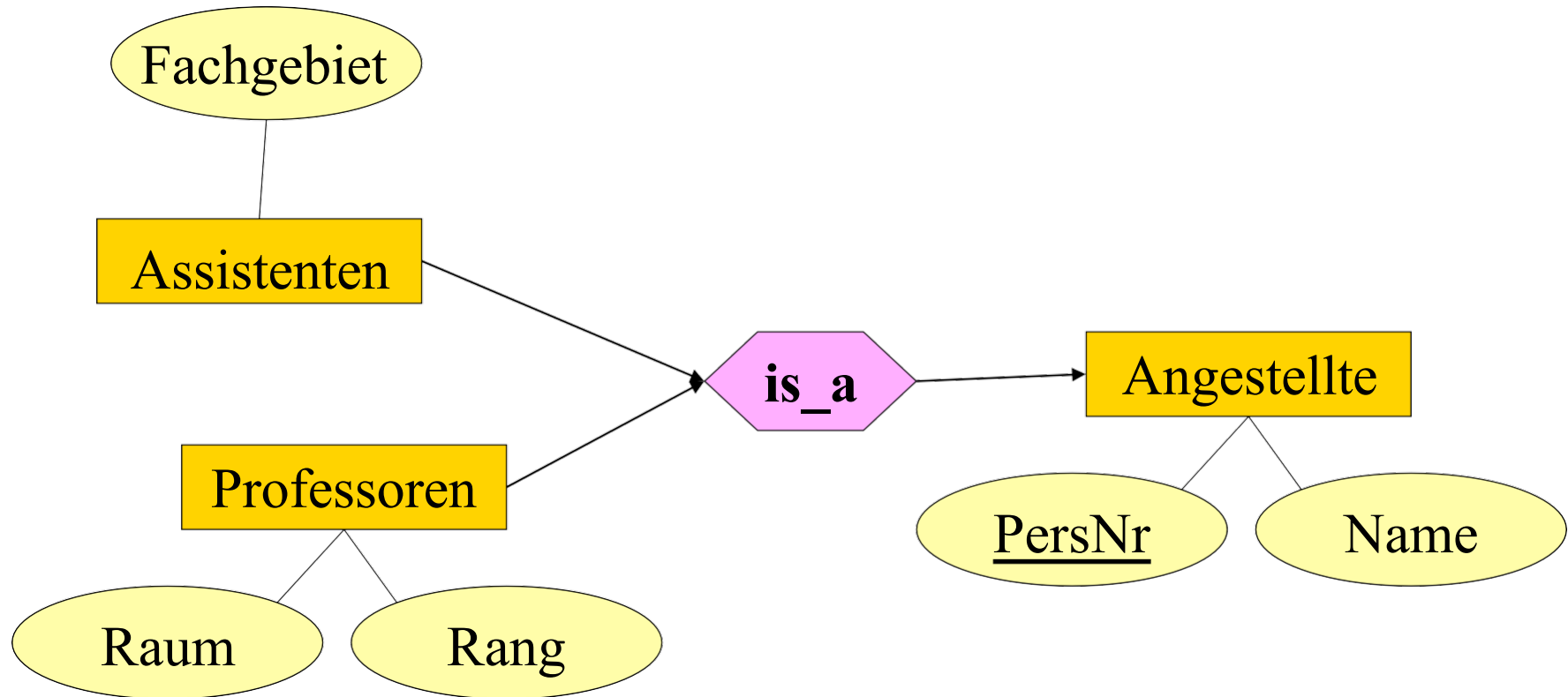
create view PruefGuete(Name, GueteGrad) as
(**select** prof.Name, **avg**(pruef.Note)
from Professoren prof **join** pruefen pruef **on**
 prof.PersNr = pruef.PersNr
group by prof.Name, prof.PersNr
having count(*) > 50) ■ ■ ■

k-Anonymität
(k hier 50)

für die Vereinfachung von Anfragen

```
create view StudProf (Sname, Semester, Titel, Pname) as  
select s.Name, s.Semester, v.Titel, p.Name  
from Studenten s, hören h, Vorlesungen v, Professoren p  
where s.MatrNr=h.MatrNr and h.VorlNr=v.VorlNr and  
v.gelesenVon = p.PersNr
```

```
select distinct Semester  
from StudProf  
where PName= `Sokrates`;
```



Angestellte: {[PersNr, Name]}

Professoren: {[PersNr, Rang, Raum]}

Assistenten: {[PersNr, Fachgebiet]}

```
create table Angestellte  
  (PersNr integer not null,  
   Name   varchar (30) not null);
```

```
create table ProfDaten  
  (PersNr integer not null,  
   Rang   character(2),  
   Raum   integer);
```

```
create table AssiDaten  
  (PersNr      integer not null,  
   Fachgebiet  varchar(30),  
   Boss        integer);
```

```
create view Professoren as  
select *  
from Angestellte a, ProfDaten d  
where a.PersNr=d.PersNr;
```

```
create view Assistenten as  
select *  
from Angestellte a, AssiDaten d  
where a.PersNr=d.PersNr;
```

Zeichnen das ER-
Diagramm als
Übung ...

Untertypen als Sichten

create table Professoren

(PersNr **integer not null,**
Name **varchar (30) not null,**
Rang **character (2),**
Raum **integer);**

create table Assistenten

(PersNr **integer not null,**
Name **varchar (30) not null,**
Fachgebiet **varchar (30),**
Boss **integer);**

create table AndereAngestellte

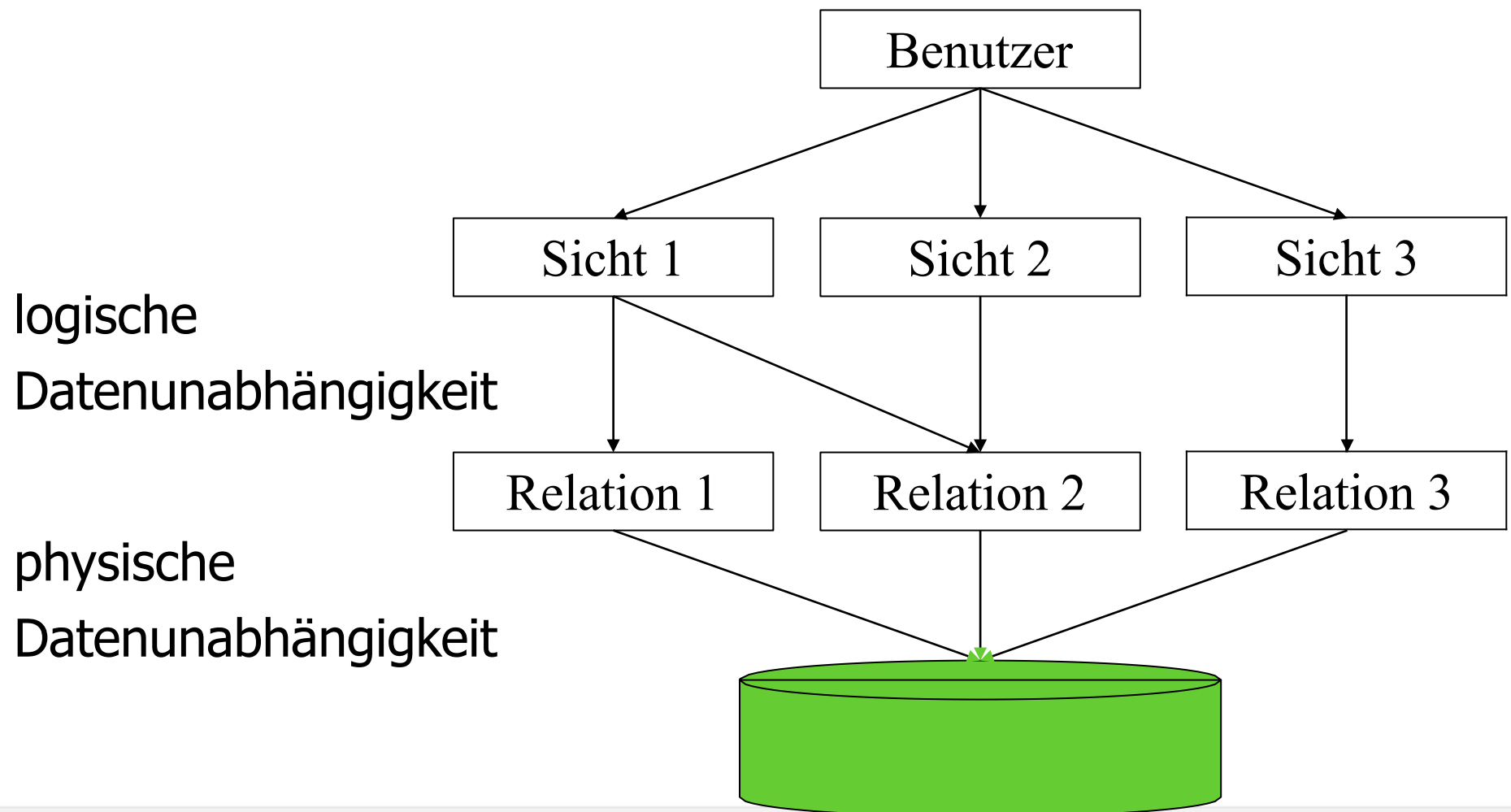
(PersNr **integer not null,**
Name **varchar (30) not null);**

```
create view Angestellte as  
    (select PersNr, Name  
    from Professoren)  
    union  
    (select PersNr, Name  
    from Assistenten)  
    union  
    (select*  
    from AndereAngestellte);
```

Zeichnen das ER-
Diagramm als
Übung ...

Obertypen als Sichten

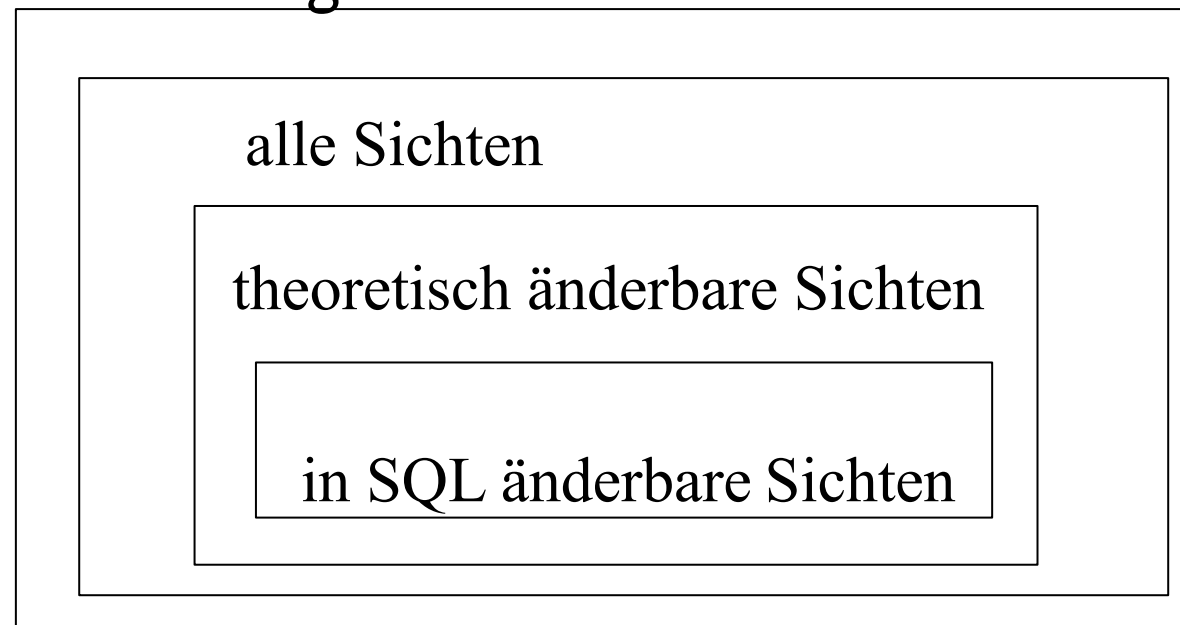
Sichten zur Gewährleistung der Unabhängigkeit von Datenformaten etc.



Beispiele für nicht änderbare Sichten

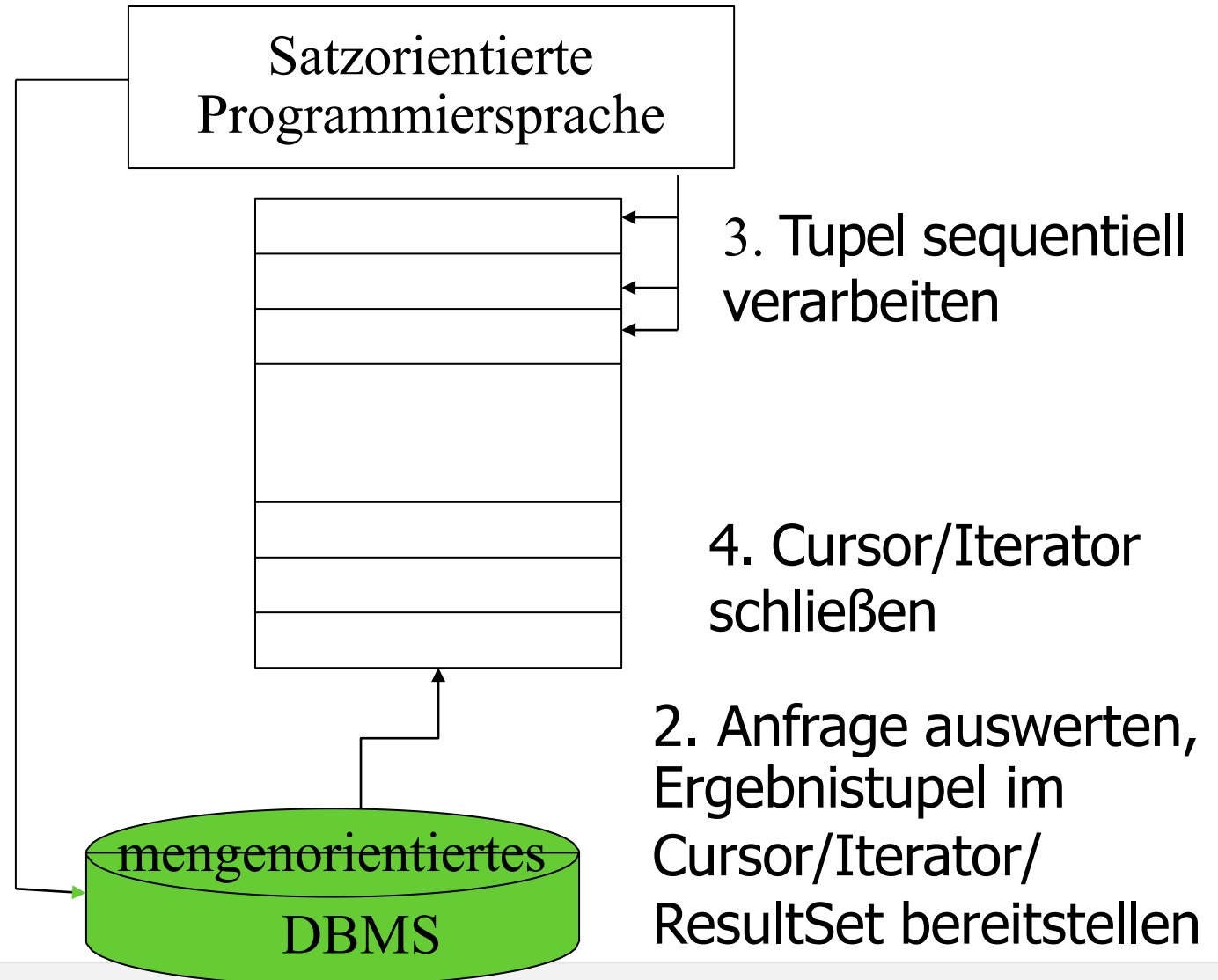
```
create view WieHartAlsPrüfer (PersNr, Durchschnittsnote) as  
    select PersNr, avg(Note)  
    from prüfen  
    group by PersNr;  
create view VorlesungenSicht as  
    select Titel, SWS, Name  
    from Vorlesungen, Professoren  
    where gelesen Von=PersNr;  
insert into VorlesungenSicht  
    values (`Nihilismus`, 2, `Nobody`);
```

- In SQL ...
- Nur eine Basisrelation
- Schlüssel muss vorhanden sein
- keine Aggregatfunktionen, Gruppierung oder Duplikateliminierung

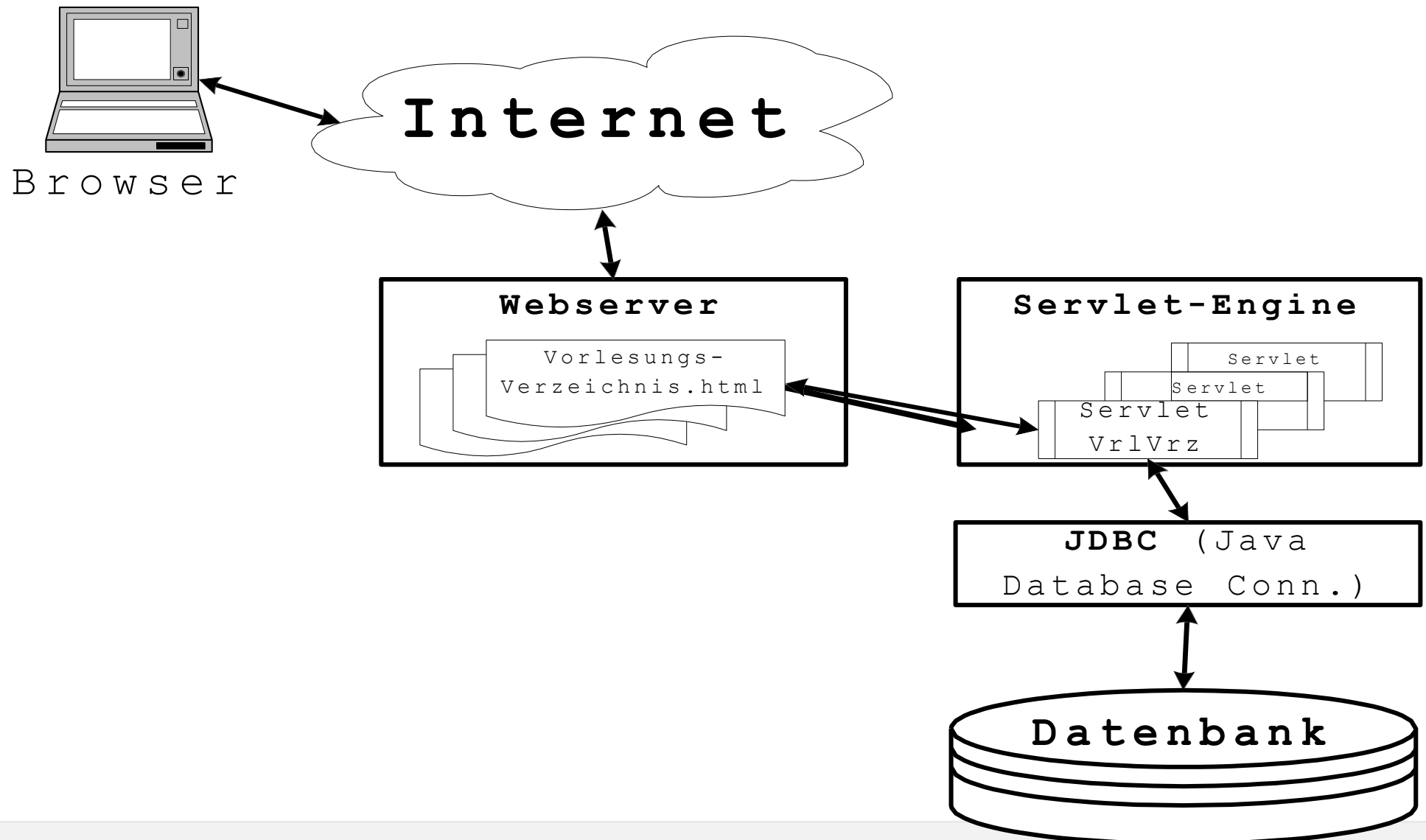


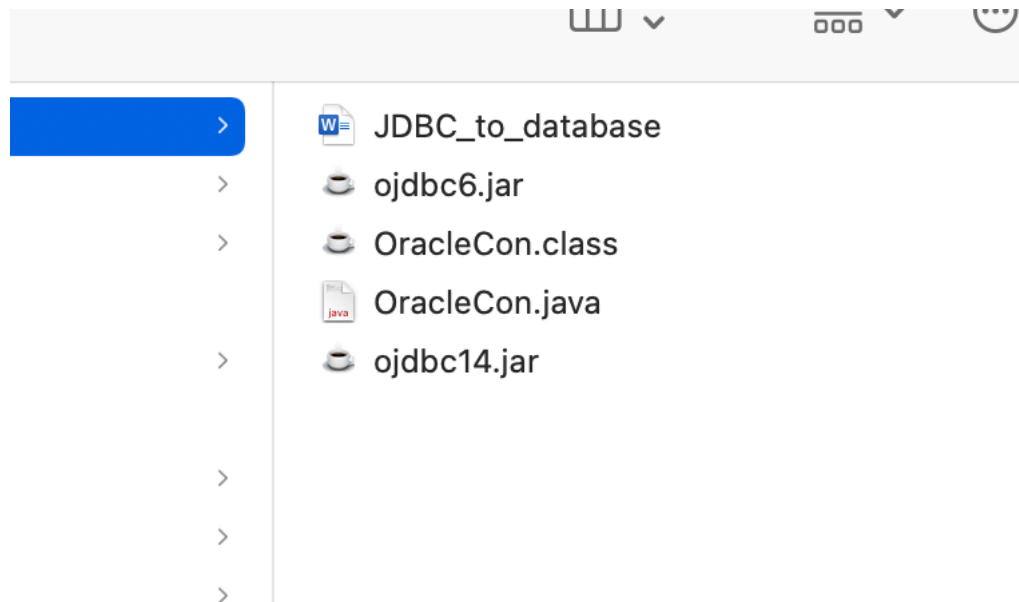
- mehrere Tupel
im Ergebnis

1. Anfrage



Web-Anbindung von Datenbanken via Servlets, JDBC, etc.





- Code
- JDBD Driver (jar)

```
Person.java RosterTest.java RegexTest.java Donate *OracleCon.java
1 import java.sql.*;
2
3 class OracleCon{
4
5     public static void main(String args[]){
6         try{
7             Class.forName("oracle.jdbc.driver.OracleDriver");
8
9             Connection con=DriverManager.getConnection(
10                 "jdbc:oracle:thin:@zeus.informatik.hs-fulda.de:1521:zeuspdb1","DBDMF20","DBDMF20");
11
12             Statement stmt=con.createStatement();
13
14             ResultSet rs=stmt.executeQuery("select * from PROFESSOREN");
15
16             while(rs.next())
17                 System.out.println(rs.getInt(1)+" "+rs.getString(2)+" "+rs.getString(3));
18
19             con.close();
20
21         }catch(Exception e){ System.out.println(e);}
22     }
23 }
24 }
```

- Code
- JDBC Driver (jar)